Before You Buy

Weekly Mini Project 03 — Team 1 · Project Specification

Project	Before You Buy
Team leader	Mulham Al Kasir
Team members	Maya Al Hamwi, Allaith Issa, Mustafa Al Shaar, Mohammad Kashmar, Rami Al Shaar

> Know what you already have, what to use first, and what you really need to buy.

1. Project Overview

Most households buy food they already have, forget what is hidden at the back of the fridge, and throw away products that expired unnoticed. A shopping list written from memory is usually wrong.

Before You Buy is a household pantry assistant. The household records what it has at home — quantity, the minimum amount it always wants to keep, and expiry dates. The application then works out, from that real stored data, which items are running low, which must be used soon, which have expired, and what belongs on the next shopping list.

This is **not** a shop. Nothing is sold, priced or paid for. The value of the product is in its **decisions**: statuses and shopping suggestions that are always derived from the stored data, never typed in by hand.

2. Team

The team leader and members are listed in the table at the top of this document.

Team Responsibility

- The leader coordinates the internal work split and makes sure the team reaches a working integrated result.
- Frontend/backend ownership is not assigned by the instructor.
- The team should decide how responsibilities are distributed.
- Every member must make a meaningful contribution.
- Every member should understand the project at a high level.
- Every member must participate in the final English presentation.

3. Project Goal

Build a web application where a household can:

- keep a pantry of items with quantities, minimum quantities, categories and expiry dates;
- record when it uses up part of an item;
- immediately see which items are **LOW STOCK**, **USE SOON**, **EXPIRED** or **OK**;
- generate a shopping list from the real state of the pantry, and add its own extra entries;
- mark shopping-list entries as purchased, which restocks the pantry correctly;
- see a dashboard that summarizes the household's current situation.

All statuses, suggestions and summaries must be calculated by the backend from the data stored in MongoDB at the moment they are requested.

4. Technology Constraints

Area	Requirement
Frontend	React + TypeScript
Backend	NestJS
Database	MongoDB, through Mongoose
API documentation	Swagger / OpenAPI

- All application data is stored in MongoDB and must survive a backend restart.
- How you organize your React app and your NestJS backend is your team's decision.
- Do **not** add user accounts, login or passwords, another database, external AI services, cloud services or paid APIs.

5. User Scenario

Rana manages the pantry for her family. On Sunday she opens Before You Buy. The dashboard shows **3 LOW STOCK**, **2 USE SOON** and **1 EXPIRED** item.

Milk has a quantity of 1, a minimum quantity of 2, and expires in two days — it is shown as both **LOW STOCK** and **USE SOON**. The yoghurt expired yesterday and is marked **EXPIRED**. Rice (quantity 4, minimum 2, no expiry date) is **OK**.

She presses "Generate shopping list". The list now contains Milk (buy 1, to reach the minimum of 2), Yoghurt (buy 3 — the expired stock no longer counts) and the other low-stock items. She also adds "Birthday candles", which is not a pantry item. Pressing the button again does not add Milk a second time.

After shopping she marks Milk as purchased (2 bought) and enters the expiry date printed on the new cartons. The pantry now shows Milk with quantity 3 and status **OK**, and the entry moves to the purchased history.

6. Required Features

Feature	Description
Pantry list	View all pantry items with quantity, minimum quantity, category, expiry date and current status(es).
Filter and search	Filter by category and by status; search by name (partial, case-insensitive). Filters can be combined.
Add / edit / remove items	Create, update and delete pantry items.
Use an item	Record that a quantity of an item was used, reducing its quantity.
Derived statuses	Show LOW STOCK, USE SOON, EXPIRED or OK for every item, calculated by the system.
Item details	One item with its status reasons (for example "2 below minimum", "expires in 2 days").
Generate shopping list	Create shopping-list entries from the current pantry state.
Manual entries	Add shopping-list entries that are not pantry items.
Mark purchased	Mark an entry as purchased; linked pantry items are restocked.
Purchased history	See purchased entries with the date they were purchased.
Dashboard	A summary of the pantry and the shopping list.
Categories	Pantry items belong to one of a set of categories; categories are managed internally.

Pantry item information

Each pantry item must have at least: name, category, quantity, unit (for example pieces, kg, litres), minimum quantity, and an optional expiry date.

Shopping-list entry information

Each entry must have at least: name, quantity to buy, whether it is linked to a pantry item, status (**Pending** or **Purchased**), the date it was created and, once purchased, the date it was purchased.

7. Business Rules

Items

1. Item names are unique in the pantry, ignoring upper/lower case and surrounding spaces ("Milk" and " milk " are the same item).
2. Quantity and minimum quantity are numbers of zero or more. Quantity may be a decimal (for example 0.5 kg).
3. An item must belong to an existing category.
4. Using an item reduces its quantity by the used amount. The used amount must be greater than zero and cannot be more than the current quantity.

Statuses

1. Statuses are **calculated by the system** and are never entered or stored by hand.
2. **LOW STOCK** — the quantity is below the minimum quantity.
3. **EXPIRED** — the item has an expiry date that is before today.
4. **USE SOON** — the item has an expiry date that is today or within the next **3 days**, and is not expired.
5. **OK** — none of the statuses above applies.
6. An item may have more than one status at the same time (for example LOW STOCK and USE SOON). EXPIRED and USE SOON never appear together.
7. Items without an expiry date can never be USE SOON or EXPIRED.
8. Statuses must be correct for **today's date**, even if the item was not edited since yesterday.

Shopping list

1. Generating the shopping list adds an entry for every item that is LOW STOCK or EXPIRED.
2. The suggested quantity is the amount needed to bring the item back up to its minimum quantity. Expired stock counts as zero. The suggested quantity is always at least 1.
3. A pantry item can have at most **one Pending** entry. Generating the list again must not create duplicates; if a Pending entry already exists, its suggested quantity is brought up to date instead.
4. Manual entries are allowed for things that are not pantry items. Their names cannot duplicate another Pending entry.
5. Marking an entry as purchased requires the purchased quantity (greater than zero). An entry can only be purchased once.
6. Purchasing a linked entry restocks the pantry item:
    - normally, the purchased quantity is **added** to the current quantity, and the user may update the expiry date;
    - if the item was **EXPIRED**, the expired stock is discarded: the new quantity **is** the purchased quantity, and a new expiry date **must** be given.
7. A pantry item that has a Pending shopping-list entry cannot be deleted until that entry is removed or purchased.

Categories

1. Categories are maintained by the household's internal tooling, not from the normal user interface (see section 13).
2. A category that is still used by any pantry item cannot be deleted.

8. Required Screens / Views

Screen	Must show / allow
Dashboard	Number of items per status; items that are EXPIRED or USE SOON, soonest first; number of Pending shopping entries; number of items per category.
Pantry	All items with status badges; the filters and search; links to item details; a way to add an item.
Item Form	Add or edit an item, with clear messages for invalid input.
Item Details	All item information, status reasons, a "use" action and the item's purchase history.
Shopping List	Pending entries (with a "generate" action, a manual-entry form, a purchase action and a remove action) and the purchased history.

All screens must be reachable through the application's navigation.

9. Required User Flows

Flow A — Track an item

1. The user adds "Milk": quantity 1, minimum 2, unit litres, category Dairy, expiry in 2 days.
2. The pantry shows Milk as LOW STOCK and USE SOON.
3. The user records using 1 litre. Milk now has quantity 0 and is still LOW STOCK.
4. The user tries to use 1 more litre and is told there is not enough left.

Flow B — Build the shopping list

1. The user presses "Generate shopping list".
2. An entry appears for every LOW STOCK or EXPIRED item, with the correct suggested quantity.
3. The user adds a manual entry ("Birthday candles", quantity 1).
4. The user generates again: no duplicates appear.

Flow C — Shop and restock

1. The user marks the Milk entry as purchased with quantity 2.
2. Milk's quantity increases by 2, and its status is recalculated.
3. The user marks the expired Yoghurt entry as purchased without an expiry date — this is rejected.
4. The user repeats with a new expiry date. Yoghurt's quantity becomes exactly the purchased amount and its status is OK.

Flow D — Find what matters

1. The user filters the pantry by category "Dairy" and status "USE SOON".
2. Only Dairy items that must be used soon appear.
3. The user searches "mil" and finds Milk.

10. Required Calculations / Derived Information

Derived information	Where it appears
Status(es) of every item	Pantry, Item Details, Dashboard
Status reasons ("1 below minimum", "expires in 2 days", "expired 3 days ago")	Item Details
Days until expiry, or days since expiry	Pantry, Item Details
Suggested quantity to buy	Shopping List
Number of items per status and per category	Dashboard
Expiring-soon list, ordered by expiry date	Dashboard
Number of Pending entries	Dashboard, Shopping List

11. Validation & Error Scenarios

General expectations

- Required information cannot be missing; numbers must be sensible.
- Invalid actions must be rejected — the system must never silently accept an invalid operation.
- The user always receives a clear, human-readable message explaining **why** an action failed.
- Something that does not exist is reported as "not found", not as an empty success.
- A request that conflicts with the current data (for example a duplicate) is reported as a conflict.
- When a request cannot be completed, the screen shows a sensible message — never a blank page.

Project-specific scenarios

Situation	Expected behavior
Item name missing, empty or only spaces	Rejected.
Item name already used (ignoring case and spaces)	Rejected as a conflict.
Negative quantity or minimum quantity, or not a number	Rejected.
Category does not exist	Rejected.
Expiry date is not a valid date	Rejected.
Using zero, a negative amount, or more than is available	Rejected; the user is told how much is available.
Opening, editing or using an item that does not exist	"Item not found".
Purchasing an entry that is already purchased	Rejected as a conflict.
Purchasing an expired item without a new expiry date	Rejected; the user is told a new expiry date is required.
Deleting an item that has a Pending entry	Rejected; the user is told to remove or purchase the entry first.
Deleting a category that is still used	Rejected as a conflict.
A request contains unexpected extra fields	Rejected.

12. Data & Persistence Requirements

- All pantry items, shopping-list entries and categories are stored in MongoDB.
- On first start with an empty database, the backend seeds realistic demo data: at least **6 categories** and **20 pantry items**, including at least 3 LOW STOCK, 3 USE SOON, 2 EXPIRED and several OK items, so every rule can be demonstrated on presentation day.

- Records are identified and linked using the **numeric, application-level id convention used in class** (as in B4F Board).

Data Design Decisions

Your team must make, implement and be able to explain these decisions:

1. **Relationship decision.** Decide how shopping-list entries relate to pantry items, and how items relate to categories: separate collections linked by id, or data stored inside another document. Write down which you chose and why.
2. **Index decision.** Choose one real, frequently used MongoDB query whose filter and sort fields are **actually stored** in your pantry item documents. Measure it with `explain("executionStats")`, choose an index only if the measurement and the query shape justify it, then measure again to prove the result. A category + expiry-date query is one reasonable candidate if it matches your implementation.
    - Do **not** create an index on the displayed LOW STOCK / USE SOON / EXPIRED / OK labels just because the interface filters by them: those statuses are calculated from quantity, minimum quantity, expiry date and today's date — they are not stored directly.
    - Follow the order: query shape → measurement → index decision → measurement again.

Both decisions must be written in your repository's README.

13. Backend Quality Requirements

- **Validation:** every request body and every filter query parameter is validated with DTOs and the validation approach taught in class. Unknown fields are rejected.
- **Meaningful errors:** failures use the correct kind of HTTP error (bad request, not found, conflict) with a clear message.
- **Logger:** log the events that matter to the business — for example items created or deleted, restocks, shopping-list generation (with how many entries were added) and rejected business-rule violations worth noticing. Do **not** log every request.
- **Swagger:** every endpoint is documented with a short summary, grouped by resource, with its request body and query parameters visible.
- **Internal-only actions:** creating, renaming and deleting **categories** is internal tooling and must be protected with the `InternalToolGuard` approach taught in class (documented in Swagger, including the required header). Everything a household member does in the normal user interface must **not** be guarded.
- **No fake logic:** statuses, suggestions and dashboard numbers must come from real calculations on stored data. Hard-coded or hand-typed values do not count.

14. Out of Scope

The following are **not required** and must not be added to the core project: prices, payments or checkout; user accounts, login, passwords or JWT; Mongoose ObjectId references or `populate`; aggregation pipelines; pagination; WebSockets; another database; external AI or recipe services; barcode scanning; deployment.

15. Minimum Deliverable (CORE)

Your project counts as complete only when **all** of the following work:

- □ A working React + TypeScript frontend and a working NestJS backend, integrated with each other.
- □ Data is stored in MongoDB and survives a backend restart; demo data is seeded.
- □ Every required screen in section 8 exists and is reachable through the navigation.
- □ Every business rule in section 7 is enforced.
- □ Items can be added, edited, used and deleted; filters and search work together.

- □ Statuses are correct for today's date, including multiple statuses at once.
- □ Shopping-list generation produces the correct entries and quantities, without duplicates.
- □ Purchasing restocks items correctly, including the expired-item rule.
- □ Categories are managed only through guarded internal endpoints.
- □ Validation, error messages, Logger and Swagger meet section 13.
- □ Both data design decisions are implemented and written in the README.
- □ The README explains how to install, configure (environment variables) and run both parts.

16. Stretch Features (optional)

Only start these after every CORE item works. They never replace a missing CORE requirement.

- **Waste report** — how many items expired in the last 30 days, per category.
- **Sorting** — sort the pantry by expiry date, name or quantity.
- **Storage location** — fridge / freezer / cupboard, with a filter.
- **Category management screen** — an internal screen that sends the internal-tool header.

17. Presentation & Demo Acceptance

Your team presents **in English**; every member speaks. Your instructor will confirm the date and time limit. Present the problem, a short product walkthrough, a live demo, your data design decisions and the most important business rules.

The live demo must pass these acceptance tests:

1. An item with quantity 1, minimum 2, expiring in 2 days shows **LOW STOCK** and **USE SOON**.
2. Using more than the available quantity is rejected with a clear message.
3. Generating the shopping list twice produces no duplicates, with correct suggested quantities.
4. Purchasing a normal item adds to its quantity; purchasing an expired item without a new expiry date is rejected, and with one replaces the quantity.
5. Deleting an item with a Pending entry is rejected.
6. In Swagger, creating a category without the internal-tool header is rejected, and with it succeeds.
7. Restarting the backend keeps all data.
8. The meaningful log lines for one restock appear in the backend console.

18. Evaluation Focus

Area	What we look for
Functional completeness	The required features and screens exist and work.
Correct business behavior	Statuses, suggestions and restocking follow the rules exactly.
Frontend/backend integration	The two parts genuinely work together on real stored data.
Backend quality	Validation, meaningful errors, Logger, Swagger and correct guard placement.
Data design	Sensible, explained relationship and index decisions.
User experience	Clear, readable screens that are easy to use.
Team collaboration and Git	Shared work, visible contributions from every member.
Explaining your own work	Every member can explain the product, its rules and its design.
Presentation quality	Clear English presentation with a working live demo.